COMPLETE OBSERVABILITY SYSTEM

Project Report | Metrics, Centralized Logging & Distributed Tracing

1. Introduction

Modern distributed applications require deep visibility into their internal states to ensure high availability and rapid troubleshooting. When performance degrades or errors occur, engineers need immediate answers to what is failing and why. This project implements a complete, containerized observability system combining the three essential pillars: metrics, logs, and distributed traces for a sample e-commerce API.

2. Abstract

The project integrates FastAPI, Prometheus, Grafana Loki, Grafana Alloy, OpenTelemetry, Jaeger, and Grafana using Docker Compose. A Python FastAPI application exposes performance metrics, structured logs, and OpenTelemetry trace headers. Prometheus scrapes application metrics, Grafana Alloy discovers Docker containers and forwards stdout logs to Loki, and Jaeger captures OTLP traces. Grafana serves as the unified visualization layer connecting all data sources.

3. Tools & Technologies Used

Tool / Technology	Purpose
Docker & Docker Compose	Containerize the application and all observability backend services onto a shared network.
FastAPI & Uvicorn	Provide the sample e-commerce REST API endpoints (/products, /orders, /slow, /error).
Prometheus	Collect and store numerical time-series metrics via pull-based scraping.
Grafana Loki & Alloy	Alloy collects container stdout logs via the Docker socket and forwards them to Loki for LogQL search.
OpenTelemetry & Jaeger	OpenTelemetry instruments the Python application to generate spans/traces, stored and visualized in Jaeger.
Grafana	Unify Prometheus, Loki, and Jaeger into a single dashboard and exploration interface.

4. Core Observability Pillars

- **Metrics:** Numerical time-series data measuring counters (HTTP request totals), gauges (active in-flight requests), and histograms (request duration percentiles).
- **Logs:** Structured text records timestamping application events, warnings, and failures (e.g., database or product validation errors).
- **Traces:** End-to-end request lifecycles broken down into timed operations (spans) to isolate latency bottlenecks.

5. Steps Involved in Building the Project

- **Step 1 - Application Development:** Created a Python FastAPI service with custom middleware to track request counts, latencies, and structured logs.
- **Step 2 - Containerization:** Wrote an optimized Dockerfile and `.dockerignore` to package the FastAPI application into a container image.
- **Step 3 - Prometheus Integration:** Configured Prometheus (`prometheus.yml`) to scrape application metrics from the `/metrics` endpoint at regular scrape intervals.
- **Step 4 - Centralized Logging:** Deployed Loki and Grafana Alloy, mounting the Docker socket so Alloy automatically discovers container streams and pushes logs to Loki.
- **Step 5 - Distributed Tracing:** Instrumented the application with OpenTelemetry SDK and OTLP HTTP exporter, routing traces directly to Jaeger.
- **Step 6 - Unified Visualization:** Configured Grafana to connect with Prometheus, Loki, and Jaeger as data sources and built real-time monitoring dashboards.
- **Step 7 - System Verification:** Generated successful product queries, simulated errors (HTTP 500), and slow requests (5s delay) to verify metrics, logs, and traces.

6. Conclusion

The project successfully demonstrates a production-grade observability pipeline where performance metrics, log aggregation, and distributed tracing work in synergy. It provided hands-on experience in container orchestration, telemetry instrumentation, PromQL/LogQL querying, and unified dashboard visualization. Future enhancements could include automated alerting rules, log-to-trace correlation IDs, and CI/CD automated health testing.

7. Project Repository

Complete source code, configuration files, and setup instructions are available on GitHub:

https://github.com/ayushshahir/smartshop-observability